PROJECT DOCUMENTATION

Appkhila

Full-Stack E-Commerce Web Application

1 Foundation	2 Full Stack	3 Catalog	4 Detail	5 Cart	6 Auth	7 Checkout

8 API	9 Orders	10 Deploy

Repository **github.com/Bhargav-0007/appkhila**

Progress **Step 2 of 10 — 20% complete**

Date **May 2026**

Author **Bhargav**

Table of Contents

1. Project Overview

Appkhila is a modern, full-stack e-commerce web application being built step-by-step across 10 milestones. The goal is a production-quality online store with product browsing, a shopping cart, user authentication, a checkout flow, order tracking, and an admin dashboard.

The project follows a clean separation of concerns: a **React + TypeScript** frontend handles all user interaction, while a **Java Spring Boot** backend serves a RESTful API backed by an **H2 relational database**. A Vite dev-proxy bridges the two during development so the frontend calls `/api/*` and never hard-codes the backend URL.

> ✅ **Current Status — 20% complete (Steps 1 & 2 done)**
> The project has a fully working frontend with routing, responsive layout, and a live Spring Boot API seeded with 30 products across 6 categories.

2. Tech Stack

Frontend

Technology	Version	Purpose
React	19	UI component model, hooks, declarative rendering
TypeScript	6.0	Static typing across all frontend code
Vite	8	Build tool — instant HMR, optimised production bundle
Tailwind CSS	3.4	Utility-first styling with custom colour palette
React Router	7	Client-side routing (`/` , `/products` , `/cart` ...)
Zustand	5	Lightweight global state for cart and UI

Backend

Technology	Version	Purpose
Java	17 LTS	Backend language
Spring Boot	3.3.6	Application framework — DI, auto-config, embedded Tomcat
Spring Web (MVC)	—	REST controllers, request mapping, JSON serialisation
Spring Data JPA	—	ORM layer — repositories, JPQL queries, pagination
H2 Database	—	File-based relational DB (persists between restarts)

Technology	Version	Purpose
Lombok	—	Reduces boilerplate: `@Data`, `@Builder`, `@RequiredArgsConstructor`
Maven	3.9	Dependency management and build tool

Architecture Decision — H2 File Mode

H2 runs in file-based mode (`jdbc:h2:file:./server/data/shopdb`) so the database survives server restarts. No external database setup is required. In Step 8, this will be upgraded to MySQL or PostgreSQL for production.

3. Complete Project Roadmap

1

Step 1 — Foundation & Home Page Done

Vite + React + TypeScript scaffold, Tailwind CSS, React Router, TypeScript interfaces, mock data, Home page with Hero / Categories / Promotions / Newsletter, Layout with Header + Footer.

2

Step 2 — Full-Stack Setup & Responsive UI Done

Spring Boot backend with Products & Categories API, H2 database, 30 seeded products. Responsive Header with search + cart badge + hamburger drawer, Zustand stores, Toast system, Skeleton loaders, ScrollToTop, Breadcrumb, Vite proxy.

3

Step 3 — Product Catalog Page Next

Product listing grid with pagination, category sidebar filter, sort options (price, rating), search results, ProductCard component with hover effects.

4

Step 4 — Product Detail Page

Full product detail view with image gallery, ratings display, stock indicator, quantity selector, Add to Cart CTA, related products, and product reviews section.

5

Step 5 — Shopping Cart

Cart page with quantity update / remove, price summary, coupon code field, cart persistence (localStorage via Zustand persist middleware), cart count badge live update.

6

Step 6 — User Authentication

Register / Login pages, JWT token handling, protected routes, user profile page, Spring Security on the backend, password hashing with BCrypt.

7

Step 7 — Checkout Flow

Multi-step checkout: address form, order review, payment UI (Stripe integration), order confirmation page, backend order creation API.

8

Step 8 — Full Backend API

Complete REST API for all entities (users, orders, reviews), upgrade to MySQL/PostgreSQL, input validation, global exception handler, API documentation (Swagger).

9

Step 9 — Order Management & Admin

Order history page, order detail & tracking, Admin dashboard with product CRUD, inventory management, sales analytics charts.

10

Step 10 — Polish, SEO & Deployment

Performance optimisation, meta tags & Open Graph, image optimisation, CI/CD pipeline (GitHub Actions), deploy frontend (Vercel) and backend (Railway / AWS).

4. Step 1 — Foundation & Home Page (10%)

Step 1 established the entire project foundation: build tooling, styling system, routing, TypeScript types, and the first fully-built page.

4.1 Folder Structure

```
ecommerce-app/
├── index.html ← SEO meta + Inter font import
├── vite.config.ts ← Vite + /api proxy
├── tailwind.config.js ← Custom colours + animations
├── package.json
└── src/
    ├── main.tsx ← React 19 entry point
    ├── App.tsx ← BrowserRouter + all Routes
    ├── index.css ← Tailwind directives + @layer components
    ├── constants/
    │   └── theme.ts ← SITE_NAME, ROUTES, NAV_LINKS
    ├── types/
    │   ├── product.ts ← Product, Category, Review interfaces
    │   ├── cart.ts ← CartItem, Cart interfaces
    │   └── user.ts ← User, Address, Order interfaces
    ├── data/
    │   └── categories.ts ← 6 mock categories (Unsplash images)
    ├── components/layout/
    │   ├── Layout.tsx ← Header + <Outlet> + Footer wrapper
    │   ├── Header.tsx ← Responsive header (Step 2 enhanced)
    │   └── Footer.tsx ← 4-column footer with links
    └── pages/
        ├── Home.tsx ← Full home page (5 sections)
        ├── Products.tsx ← Placeholder → Step 3
        ├── Cart.tsx ← Placeholder → Step 5
        ├── Login.tsx ← Placeholder → Step 6
        ├── Register.tsx ← Placeholder → Step 6
        └── NotFound.tsx ← 404 page
```

4.2 Configuration Files

tailwind.config.js — custom design tokens

Token	Values	Used for
`colors.primary`	50 → 900 blue scale	Buttons, links, hero background
`colors.accent`	Orange 400/500/600	CTA buttons, cart badge
`fontFamily.sans`	Inter, system-ui	All body text
`animation.slide-in`	opacity + translateY 0.25s	Toast notifications (Step 2)

4.3 TypeScript Interfaces (types/)

Interface	File	Key Fields
`Product`	product.ts	id, name, slug, price, originalPrice, images[], category, brand, rating, reviewCount, stock, featured
`Category`	product.ts	id, name, slug, image, description, productCount
`Review`	product.ts	id, userId, productId, rating, title, body, createdAt
`CartItem`	cart.ts	product: Product, quantity: number

Interface	File	Key Fields
`Cart`	cart.ts	items, total, itemCount
`User`	user.ts	id, name, email, avatar, role, createdAt
`Address`	user.ts	fullName, line1, line2, city, state, postalCode, country, isDefault
`Order`	user.ts	id, userId, items, total, status, address, createdAt

4.4 Routing & Layout

All routes share a single `Layout` component (via React Router's `<Outlet>`), which ensures the Header and Footer appear on every page consistently.

Route	Component	Status
`/`	Home.tsx	Built
`/products`	Products.tsx	Step 3
`/products/:slug`	ProductDetail.tsx	Step 4
`/cart`	Cart.tsx	Step 5
`/checkout`	Checkout.tsx	Step 7
`/login`	Login.tsx	Step 6
`/register`	Register.tsx	Step 6
`*`	NotFound.tsx	Built

4.5 Home Page — 5 Sections Explained

1. Hero Section

Full-width gradient banner (primary-700 → primary-500). Left: headline "Shop Smarter, Live Better", subtitle, two CTA buttons (Shop Now in accent orange, Browse Categories in outlined white), and 3 quick stats (50K+ Products, 1M+ Customers, 4.9★ Rating). Right: staggered 2×2 grid of product images (hidden on mobile).

2. Features Bar

4 trust badges in a responsive grid: Free Shipping (🚚), Easy Returns (🔄), Secure Payments (🔒), 24/7 Support (💬). Each has an emoji icon, bold title, and short description. Background turns `primary-50` on hover.

3. Categories Grid

6 category cards in a responsive grid (2 cols mobile → 6 cols desktop). Each shows a circular Unsplash image, category name, and product count. Clicking navigates to `/products?category=<slug>`. Image scales up and a blue ring appears on hover.

4. Promotional Banners

Two side-by-side gradient cards. Left: purple/indigo — "Up to 50% off Electronics". Right: rose/orange — "Fresh Fashion Drops". Each has a badge, heading, subtext, and a Shop link.

5. Newsletter Section

Dark (`primary-900`) section with an email subscription form. Form submission is prevented in the frontend (backend integration in Step 8).

4.6 Reusable CSS Classes (@layer components)

Class	What it produces
`.btn-primary`	Blue filled button with hover darken
`.btn-secondary`	White button with blue border
`.btn-accent`	Orange button for primary CTAs
`.card`	White rounded card with shadow and border
`.container-max`	Centred max-w-7xl with responsive horizontal padding
`.section-title`	Large, bold, centred heading
`.section-subtitle`	Muted, centred subtitle below a heading
`.badge`	Inline pill with rounded-full shape

💡 **Why Tailwind @layer components?**

These classes are defined in `@layer components` in `index.css`, which means they can be overridden by utility classes and are purged correctly in production. You get the convenience of semantic class names without losing Tailwind's flexibility.

5. Step 2 — Full-Stack Setup & Responsive UI (20%)

5.1 Spring Boot Backend Architecture

The backend is a standalone Maven project inside the `server/` folder. It runs on port **3001** and is completely independent of the React frontend.

```
server/ ├── pom.xml ← Spring Boot 3.3.6 + JPA + H2 + Lombok └──
src/main/java/com/appkhila/server/ ├── ServerApplication.java ← @SpringBootApplication entry
point ├── DataSeeder.java ← CommandLineRunner — seeds DB on first run ├── model/ │ ├──
Category.java ← @Entity: id, name, slug, description, imageUrl, productCount │ └── Product.java
← @Entity: id, name, slug, price, brand, rating, stock … + @ManyToOne Category ├── repository/ |
├── CategoryRepository.java ← JpaRepository + findBySlug │ └── ProductRepository.java ← search()
JPQL + findByFeaturedTrue ├── controller/ │ ├── CategoryController.java ← GET /api/categories,
/api/categories/{slug} │ └── ProductController.java ← GET /api/products (paginated), /featured,
/{slug} ├── dto/ │ └── ProductResponse.java ← Maps Product → JSON (images list, nested category)
└── config/ └── CorsConfig.java ← Allows http://localhost:5173
```

JPA Entity Relationships

Entity	Table	Key Fields	Relationship
`Category`	`categories`	id (UUID), name, slug, description, imageUrl, productCount	—
`Product`	`products`	id (UUID), name, slug, price, originalPrice, brand, rating, reviewCount, stock, featured, images (TEXT)	@ManyToOne → Category

> **Image Storage Strategy**
> Product images are stored as a comma-separated string in the `images` TEXT column. The `ProductResponse` DTO splits this string into a `List<String>` before sending the JSON response. In Step 8 this will be replaced with a proper image table.

JPQL Search Query

The `ProductRepository.search()` method uses a custom JPQL query to support optional category filtering and text search simultaneously:

```
@Query("SELECT p FROM Product p WHERE " +
       "(:category IS NULL OR p.category.slug = :category) AND " +
       "(:q IS NULL OR LOWER(p.name) LIKE LOWER(CONCAT('%', :q, '%')))")
Page<Product> search(
```

```
    @Param("category") String category,
    @Param("q") String q,
    Pageable pageable
);
```

Passing `null` for either parameter makes that filter inactive, so the same query handles all combinations: all products, by category, by search, or both.

5.2 Database & Data Seeding

`DataSeeder` implements `CommandLineRunner`, which Spring Boot calls automatically after the application context starts. It checks `categoryRepository.count() > 0` before inserting — so it only seeds once.

Seeded Data — 6 Categories, 30 Products (5 per category)

Category	Slug	Sample Products
Electronics	`electronics`	Wireless Headphones, 4K TV, Gaming Keyboard, USB-C Hub, Bluetooth Speaker
Fashion	`fashion`	White Sneakers, Denim Jeans, Puffer Jacket, Crossbody Bag, Sunglasses
Home & Living	`home-living`	Coffee Maker, Scented Candles, Cutting Board, LED Desk Lamp, Plant Pots
Sports	`sports`	Dumbbell Set, Yoga Mat, Water Bottle, Running Shoes, Resistance Bands
Books	`books`	Atomic Habits, The Pragmatic Programmer, Sapiens, Clean Code, The Great Gatsby
Beauty	`beauty`	Vitamin C Serum, Face Moisturiser, Matte Lipstick, Jade Roller, SPF 50

application.properties — Key Settings

```
# Server
server.port=3001

# H2 file-based database
spring.datasource.url=jdbc:h2:file:./server/data/shopdb
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.hibernate.ddl-auto=update       # creates tables if missing, updates schema

# H2 web console (dev only)
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

5.3 REST API Endpoints

Categories

GET	`/api/categories`	Returns all 6 categories as a JSON array
GET	`/api/categories/{slug}`	Returns a single category by its slug (e.g. `electronics`)

Products

GET	/api/products	Paginated list — params: `category`, `q`, `page`, `size`, `sortBy`, `sortDir`
GET	/api/products/featured	Returns all products where `featured = true` (no pagination)
GET	/api/products/{slug}	Single product by slug — 404 if not found

Example API Response — GET /api/products?category=electronics&page=0&size=2


```
{
  "content": [
    {
      "id": "a1b2c3...",
      "name": "Wireless Noise-Cancelling Headphones",
      "slug": "wireless-headphones",
      "price": 89.99,
      "originalPrice": 129.99,
      "images": ["https://images.unsplash.com/..."],
      "category": { "id": "...", "name": "Electronics", "slug": "electronics" },
      "brand": "Sony", "rating": 4.7, "stock": 45, "featured": true
    }
  ],
  "totalElements": 5,
  "totalPages": 3,
  "page": 0,
  "size": 2
}
```

5.4 Responsive Header & Navigation

The Header was completely rebuilt in Step 2. It adapts to three screen sizes:

Screen	What's Shown
Mobile (< 768px)	Logo, search icon toggle, cart icon with badge, hamburger button → slide-out drawer
Tablet (≥ 768px)	Logo, nav links, search bar, cart badge, Sign In button
Desktop (≥ 1024px)	Full layout — logo, nav links, inline search bar (max-w-sm), cart, Sign In

Mobile Drawer Behaviour

When the hamburger is tapped, a 72-wide drawer slides in from the right with: a semi-transparent black backdrop (clicking closes it), the site logo, all nav links with active highlighting, and Sign In / Create Account buttons pinned to the bottom.

Search Behaviour

On submit, the search navigates to `/products?q=<encoded query>`. This will be consumed by the Products page filter in Step 3. The search input is auto-focused when the mobile search toggle is tapped.

5.5 State Management (Zustand)

cartStore — Persisted to localStorage

State / Action	Type	Description
`items`	`CartItem[]`	Array of {product, quantity} objects
`addItem(product, qty?)`	Action	Adds new item or increments existing quantity
`removeItem(id)`	Action	Removes a product entirely from the cart
`updateQuantity(id, qty)`	Action	Sets quantity; calls removeItem if qty ≤ 0
`clearCart()`	Action	Empties all items
`itemCount()`	Computed	Total number of items (sum of quantities)
`total()`	Computed	Sum of price × quantity for all items

uiStore — UI-only state (not persisted)

State / Action	Description
`mobileMenuOpen`	Boolean — controls drawer visibility
`toggleMobileMenu()`	Flips open/closed

State / Action	Description
`closeMobileMenu()`	Forces closed (called on nav link click)
`toasts: Toast[]`	Queue of active notification toasts
`addToast(type, message)`	Adds toast and auto-removes it after 3.5s
`removeToast(id)`	Manually dismisses a toast

5.6 UI Component Library

Toast / ToastContainer

Renders a stacked list of dismissible notification cards in the bottom-right corner. Supports 4 types: `success` (green), `error` (red), `info` (blue), `warning` (yellow). Each toast auto-dismisses after 3.5 seconds and has a manual close button. Animated in with the `slide-in` Tailwind keyframe.

Skeleton / ProductCardSkeleton / ProductGridSkeleton

Animated pulse placeholder components shown while data loads. `ProductCardSkeleton` matches the exact shape of a product card (image + 3 text lines). `ProductGridSkeleton` renders N cards in the correct grid layout. Used in Step 3 onwards when fetching products from the API.

ScrollToTop

A circular blue button that appears fixed in the bottom-left corner after the user scrolls down 300px. Clicking it smooth-scrolls back to the top. Uses a passive scroll event listener for performance.

Breadcrumb

Accepts an array of `{label, href?}` objects and renders `Home / Products / Product Name` with clickable intermediate links. The last crumb is always bold and non-clickable.

🔗 **Vite Proxy — How the Frontend Reaches the Backend**

In `vite.config.ts`, all requests to `/api/*` are proxied to `http://localhost:3001`. This means React code simply calls `fetch('/api/products')` without knowing the backend URL — making it trivial to point at a different server in production.

6. How to Run the Project

Prerequisites

Tool	Required Version	Check Command
Node.js	18+	`node --version`
Java	17+	`java -version`
npm	9+	`npm --version`

Step-by-step Setup

```
# 1. Clone the repository
git clone https://github.com/Bhargav-0007/appkhila.git
cd appkhila

# 2. Install frontend dependencies
npm install

# 3. Start both servers together (recommended)
npm run dev:all

# — or start them separately —
npm run dev          # Frontend → http://localhost:5173
npm run dev:server   # Backend  → http://localhost:3001
```

Available URLs

URL	What it serves
`http://localhost:5173`	React frontend (Vite dev server)
`http://localhost:3001/api/categories`	Categories JSON API
`http://localhost:3001/api/products`	Products JSON API (paginated)
`http://localhost:3001/api/products/featured`	Featured products
`http://localhost:3001/h2-console`	H2 database web console

> 💡 **H2 Console Connection Settings**
>
> Open `http://localhost:3001/h2-console` and use: JDBC URL: `jdbc:h2:file:./server/data/shopdb` · Username: `sa` · Password: (empty)

7. What's Coming in Step 3

3

Product Catalog Page — The full products listing experience

The Products page will be built out completely with these features:

Feature	Details
Product Grid	Responsive 2/3/4 column grid with ProductCard components showing image, brand, name, price, rating stars, and Add to Cart button
Category Sidebar	Filter products by category — clicking updates the URL and fetches from `/api/products?category=slug`
Search Integration	Search query from the Header search bar auto-populates and filters the product list via `?q=`
Sort Options	Sort by: Price Low→High, Price High→Low, Rating, Newest
Pagination	Previous / Next + page numbers wired to the Spring Boot paginated API
Loading Skeletons	ProductGridSkeleton shown while API fetch is in progress
Empty State	Friendly "No products found" illustration if filters return zero results

🚀 **Repository**

All code is available at **github.com/Bhargav-0007/appkhila**. The `main` branch always reflects the latest completed step. Each step is committed separately with a detailed commit message.